

Conversational Shopping Agent — How It Works Now

TechJam 2026 • Track 4 • **main** @ elimination-scan + tuned BM25 • public score **0.898** (hit@10 0.99, MRR 0.82, MTTC 3.15) • no LLM, no network

1. The 60-second model

Three separate things. The “customer” is a script, not an LLM.

- **Test case** (one line of `public_set.jsonl`): a hidden target `parent_asin`, a `scenario_type` (buying / browsing / intent_override / boundary), and an anonymised `user_profile`. Nothing else.
- **Catalog**: 50,000 frozen products. The agent searches it; it is never told which one is the target.
- **Simulator** (in the evaluator): builds a hidden **intent card** of ~4 constraint phrases from the target's own features/details, then role-plays a shopper who reveals **≤2 of them per turn — only when the agent's reply sets ask_attribute**. It discloses everything matching the asked attribute, or anything if the agent asked other.

The session ends the instant the target appears in a shown list (**rank + turn freeze there**), or after turn 10 (a miss). Wrong recommendations cost nothing but the turn.

2. One turn, end to end

```
reset(session_id, user_profile) -> DialogState created

respond(session_id, user_message, turn, top_k=10):

    S3 state.observe(turn, text)      append utterance; turn-1 text -> opening
    |                                reversal cue -> apply_override(): down-weight prior turns to 0.35
    |                                'no preference' -> mark that attribute dead
    v
    S2 classify(opening) -> Route      buying vs browsing; only route.tone (question wording) is used
    v
    S5 retrieve(index, state)          terms route : FTS5 BM25 over the WHOLE conversation, pool 300
    |                                field weights (title 8, categories 5, features 6, details 6,
    |                                store 0.5, description 0.25)
    |                                focused route: same, post-override turns only (only after an override)
    |                                fused by reciprocal-rank fusion (k=60)
    v
    S6 rerank(index, state, pool)      top 200 rescored:
    |                                span_coverage (verbatim phrase matches - dominant)
    |                                + bm25 / max_bm25 (normalised retrieval score)
    |                                + 0.02 * popularity (tie-break only)
    v
    S4 policy.select(state)            FixedPolicy: ask 'other' until the customer refuses it, then walk
    |                                feature -> use_case -> style -> material -> color -> size -> category
    v
    S7 _shortlist(state, ranked, turn) which 10 to show this turn - see section 3
    v
    message = route.tone + question(attribute)
    return {message, ask_attribute, recommendations, usage:{0,0}}

(whole path wrapped in try/except -> valid empty response on any error)
```

Retrieval and reranking run **every turn** against the full accumulated state, so each turn's ranking reflects everything disclosed so far, including a post-override reversal.

3. When does it show a list? (recommendation timing)

turn	what happens
1 - 2	No list (questions only) — unless the confidence gate fires: top candidate leads the pool by $\geq 20\%$ ($(\text{best-runner_up})/\text{best} \geq 0.20$) and $\text{turn} \geq 2$. Then it shows the top 10 early.
3	First scheduled list = the top 10 of the re-ranked pool (<code>first_recommend_turn = 3</code>).
4 - 10	Elimination scan . Every product shown and not hit on is a confirmed non-target (the session would have ended), so each turn: drop everything already shown, show the top 10 of the re-ranked <i>survivors</i> . Turn 4 → ranks 11-20, turn 5 → 21-30, ... up to ~rank 80.

Because rerank runs every turn, a target that drifts down the ranking is still a survivor and surfaces a turn later — no gap.

Exception for intent override: a list shown *before* the override confirms nothing (the evaluator ignores hits until the

override lands), so the shown-set is cleared on override detection and the scan restarts on the post-override ranking. `hold_until_stalled` is off.

4. What the agent does each round, by scenario

All four run the same pipeline. They differ only in the opening line, one scripted twist, and therefore in *when* the four constraints land.

4a. Buying (40% of sessions • opens with one firm requirement)

turn	customer	agent this turn
1	"looking for {category}. A key requirement is: C1."	store C1; retrieve+rerank on {category, C1}; ask other; no list
2	"what matters: C2; C3"	retrieve+rerank on {cat, C1-C3}; ask other; list <i>only</i> if the confidence gate fires
3	"what matters: C4"	all 4 constraints; show top 10 → HIT if target is there
4	"no additional preference for other"	retire other → ask feature; scan shows ranks 11-20
5-10	"no preference for feature / use_case / ..."	walk the ladder; scan shows 21-30, 31-40, ... 71-80

4b. Browsing (40% • opens vague)

turn	customer	agent this turn
1	"looking for {category}, but I'm still exploring."	only the category — weak retrieval; ask other; no list
2	"what matters: C1; C2"	retrieve+rerank on {cat, C1, C2}; ask other; no list (turn < 3) unless confident
3	"what matters: C3; C4"	all 4 constraints; show top 10 → HIT if there
4-10	"no preference ..."	ladder walk; elimination scan 11-20, 21-30, ...

Difference from buying: one fewer constraint on turn 1, so browsing is ~one turn behind on information. The router picks the "To point you in the right direction:" phrasing (cosmetic — the simulator ignores it).

4c. Intent override (15% • reverses a preference on turn 3 or 4)

turn	customer	agent this turn
1	"looking for {category}. {OLD preference}"	store OLD; retrieve on {cat, OLD}; ask other; no list
2	"what matters: C1; C2"	retrieve+rerank; may show a list — but hits do not count yet
3 or 4	"Actually, ignore my earlier preference. What I need is: NEW." (evaluator-injected)	<code>apply_override</code> : mark <code>override_turn</code> , down-weight prior turns to 0.35, activate the focused retrieval route; clear the shown-set, restart the scan ; show top 10 of the post-override ranking — hits now count
after	"what matters: C3; C4", then "no preference ..."	two retrieval routes (all-turns + post-override); ladder walk; elimination scan 11-20, 21-30, ...

No hit is registered before the override message arrives, so the effective session is turns 3/4-10.

4d. Boundary (5% • has no opinion on the first thing asked)

turn	customer	agent this turn
1	"looking for {category}, but I'm still exploring."	only the category; ask other; no list
2	"I don't have a preference for other; please use your judgment." (fires once)	mark other dead → ask feature; no list
3	"what matters: {constraints that classify as feature}"	retrieve+rerank; ask <code>use_case</code> ; show top 10
4-10	answers or declines each ladder rung; a discriminating constraint (e.g. a fabric blend) may only appear when the agent finally asks <code>material</code>	elimination scan re-ranks every turn, so a late constraint still surfaces the target; 11-20, 21-30, ...

The mandatory refusal burns the broad other question and retires it, so extraction runs through the slower ladder and the deciding constraint often lands at turn 5-7. The per-turn re-rank is what recovers these.

5. Current settings (reference)

stage / knob	value	file
FTS5 field weights (title, categories, features, details, store, description)	8, 5, 6, 6, 0.5, 0.25	<code>src/index.py</code>
retrieval pool size	300	<code>src/retrieval.py</code>

stage / knob	value	file
RRF constant k	60	src/retrieval.py
focused-route weight (post-override)	0.8	src/retrieval.py
rerank depth / span weight / length bonus / popularity weight	200 / 1.0 / 0.12 / 0.02	src/rerank.py
pre-override utterance weight	0.35 (largely inert - see note)	src/state.py
clarification policy	FixedPolicy: 'other', then feature-ladder	src/policy.py
first_recommend_turn / confidence margin / earliest turn	3 / 0.20 / 2	starter/agent.py
elimination_scan / hold_until_stalled	on / off	starter/agent.py

Note on the 0.35 override weight: `full_text()` (the main retrieval route) and `query_spans()` (the reranker) both ignore utterance weight, so the down-weight only decides which turns the *focused* route sees. In this evaluator the retracted preference is still a real constraint of the target, so nothing breaks; a private simulator using a genuine decoy would expose this.

6. Where it stands

split / scenario	sessions	hit@10	MRR	MTTC
public (all)	200	0.990	0.821	3.15
buying	80	0.988	0.830	3.00
browsing	80	0.988	0.826	2.91
intent_override	30	1.000	0.755	3.90
boundary	10	1.000	0.900	4.00
dev (tuned on) / holdout (untouched)	120 / 80	0.98 / 1.00	0.84 / 0.80	3.1 / 3.2

Score progression: weak BM25 baseline 0.107 → teammate pipeline 0.859 → + elimination scan 0.893 → + tuned BM25 weights **0.898**. Only 2 misses on the public set. 39/39 tests pass.

Full detail: `docs/ideas.md` (recommendation strategy), `docs/hard_cases.md` (failure analysis), `ARCHITECTURE.md`, `IMPLEMENTATION.pdf` (stage rationale).